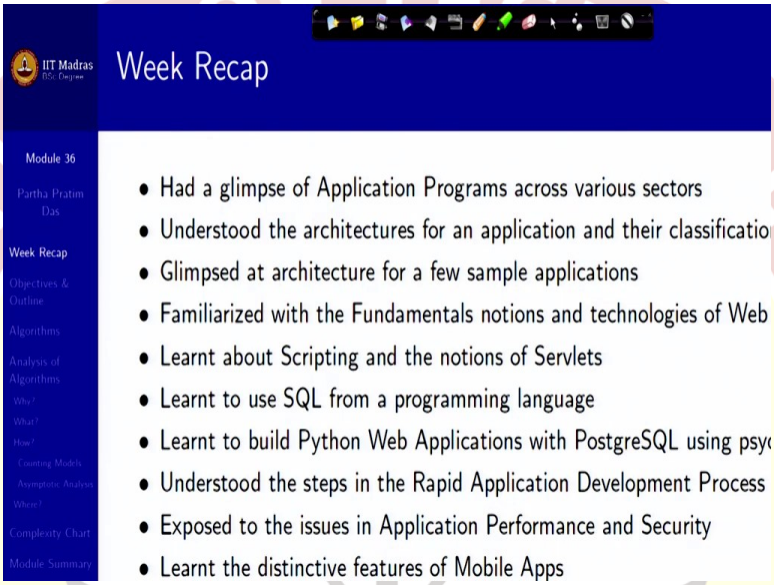
IIT Madras

ONLINE DEGREE

Database Management System
Professor Partha Pratim Das
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur
Algorithms and Data Structure/1: Algorithms and Complexity Analysis

Welcome to Module 36, Week 8 of Database Management Systems course in IIT Madras online B.Sc. program.

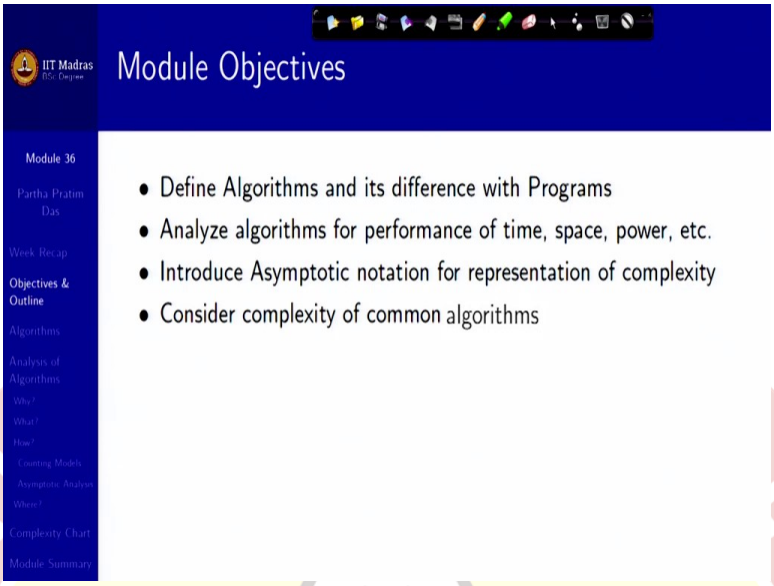
(Refer slide Time: 00:32)



In the last week, we took a wide look into how to develop applications based on databases. Using various programming languages, tools, frameworks we tried to give you an overview of the rapid development process, what it needs to do applications on the web as well as on the mobile.

As I told earlier, we would now again step back into the core of database design and if you recollect what we have primarily done in the earlier weeks, we have started by focusing on the logical design. We talked about two primary design aspects. One is the logical design, which is conceptual, which deals with entities relationships, different constraints, their representation, the algebra, calculus, everything. And the other aspect is a physical design, as to how will the bits and bytes be organized so that you can have an efficient storage and an efficient access. So, it is now time to start discussing that.

(Refer Slide Time: 01:49)

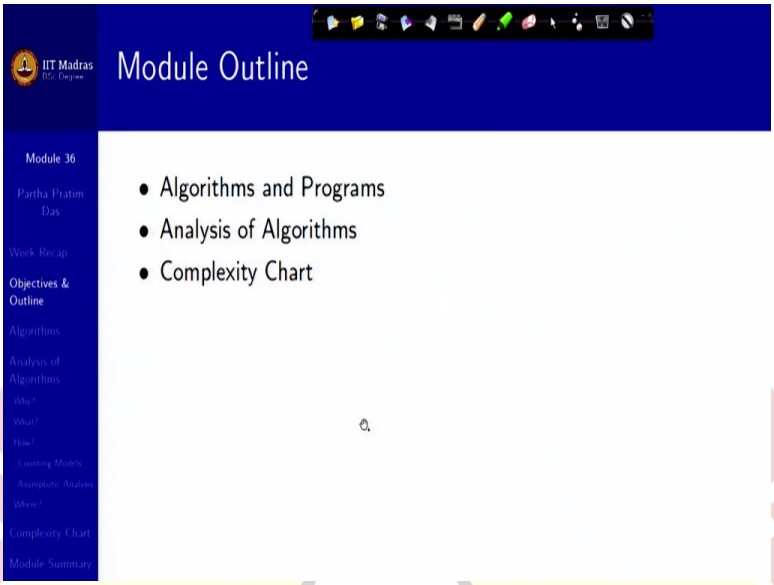


For that we will have to study some of the storage options, or not some maybe a wide range of storage options, that have evolved over time and what is available today. We have to study how in this typical storage you can organize the data, how you can make the accesses and storage efficient by things like indexing and so on.

So, keeping that target in mind I just thought that it will be good to give a little bit of recap to you on what is efficiency, what we mean by efficiency of access, what we mean by efficiency of storage, how can we compare different ways of doing accesses, different ways of organizing data in terms of its access time, in terms of its storage requirement.

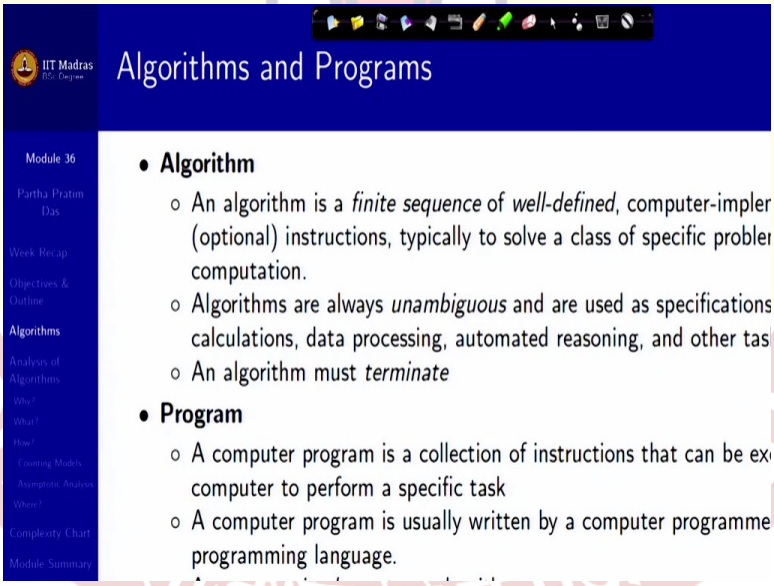
This, truly this is, these are topics in the algorithms course which you may have already studied. So, I have kept it very brief over the first 3 modules of this week, and I will start today by defining just, I mean, most of it will be recapitulation for you but just putting in context from the algorithms course as to, and the data structure course as to what we will frequently need in the design of the physical databases. So, we will start by looking at algorithms and programs and then the different issues of analysis and particularly coming to the asymptotic notation and some complexity of common algorithms.

(Refer Slide Time: 03:31)



So, this is uh the module outline, the points on the left.

(Refer Slide Time: 03:35)



So, as you know, formally an algorithm is a finite sequence of well-defined steps. The sequence has to be finite and every step has to be well defined, unambiguous to solve a class of specific problems, or to perform a computation. You want to achieve a task, so you define a sequence of instructions which are already known to you and those which are unambiguous and well defined, and by doing those steps one by one, maybe in just a sequential order or maybe repeating them, you will be able to achieve that task.

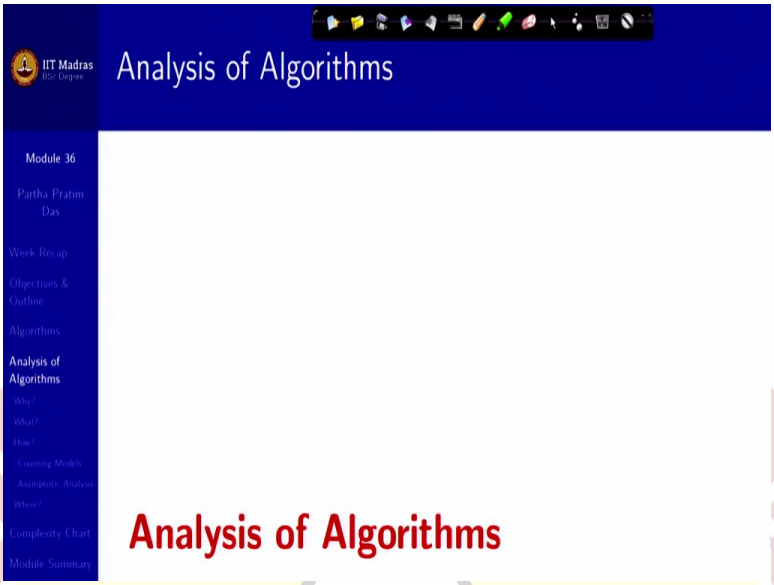
Now, more often the instructions are computer implementable but we also write algorithms which are more like for human understanding and you can find an equivalent representation algorithm which is computer implementable. But the key point about an algorithm is, and this question is often asked in the interview as to what is the difference between an algorithm and a program, the key point of an algorithm is it must terminate.

An algorithm given an input must give me the output in finite time. That time may be very long, that time may be milliseconds, it could be seconds, could be hours, months or even years, millenniums, but it provably must terminate. That is a key point.

In contrast, a program is a collection of instructions very similar to algorithms which are necessarily to be executed by a computer. That is the reason the computer programmers write programs in a certain computer language, so we say that programs implement algorithms. And the characteristics of a program which would often distinguish it from an algorithm is the fact that a program may or may not terminate.

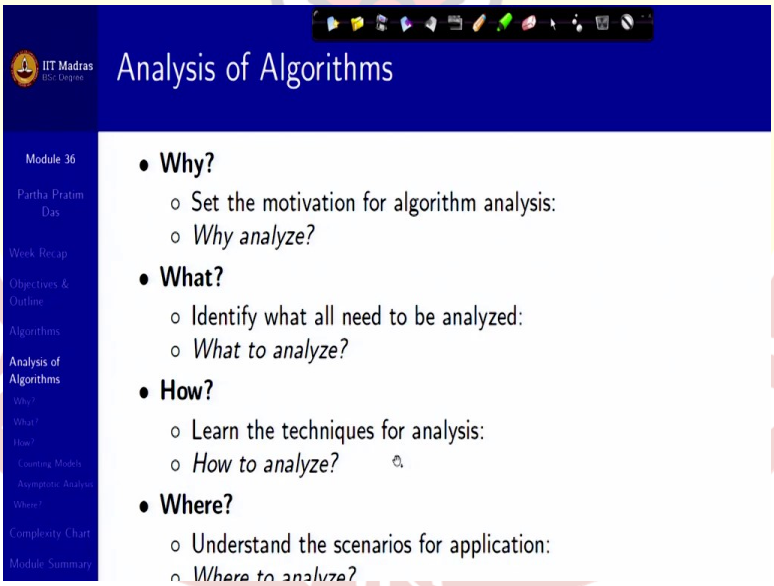
Now, for example you will ask, I mean, how is it that a program may not terminate. It is quite possible, for example think about the operating system working in your desktop or in your mobile phone. They never end. If the operating system ends, the system is dead. So, they, as long as you have the computer system, you have an operating system running. Your database system does not stop. It will have to keep running. So, these are programs which do not terminate, but they embody algorithms which must terminate.

(Refer Slide Time: 06:15)



That is the reason, mostly we talk of algorithms only and programs come in terms of realizing what the algorithm is trying to do.

(Refer Slide Time: 06:21)

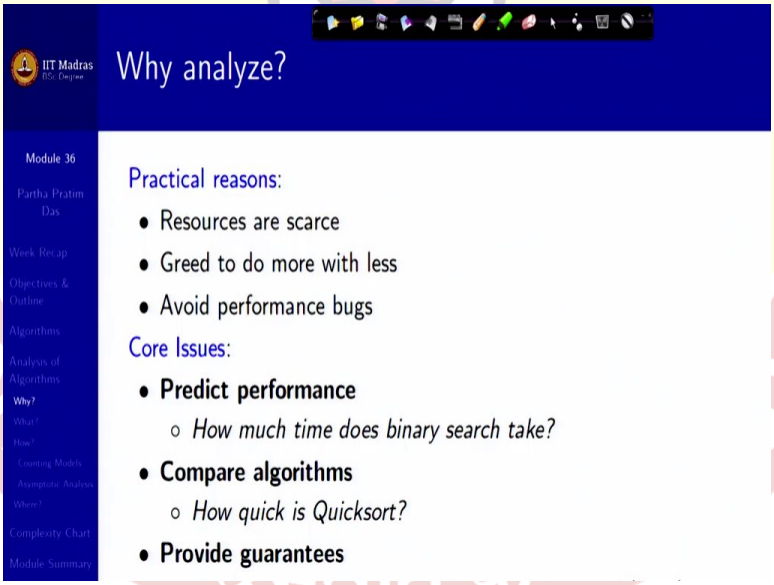


Now, the question is, we often know about analysis of algorithms, we often talk about analysis of algorithms. So, there are a couple of w questions on this which is very typical as to why should you analyze, what is the motivation, why do you actually analyze an algorithm. Then the question is what do you analyze, what identify, what all factors that need to be analyzed.

Once you have done that, then you need certain techniques, tools to analyze. So, you try to answer how to analyze. Then you ask the question where to analyze, what are the scenarios, would you analyze it for all possible inputs or for some sample inputs, what are the scenarios, where do you identify the scenarios to analyze, that is, where to analyze. And the last question is, would you keep on analyzing an algorithm all the time or there is a point when you stop analyzing algorithms and finding newer algorithms because you may be able to realize whatever best is even theoretically possible, that is when to analyze.

Now, leaving aside when to analyze because it involves quite a lot of theory in terms of lower bounds and you know NP completeness, and all that, so which is not relevant for our database discussion. You must have studied them in the algorithm course. I would try to take you quickly through these four points which you, I am sure know in some way or other already.

(Refer Slide Time: 07:50)

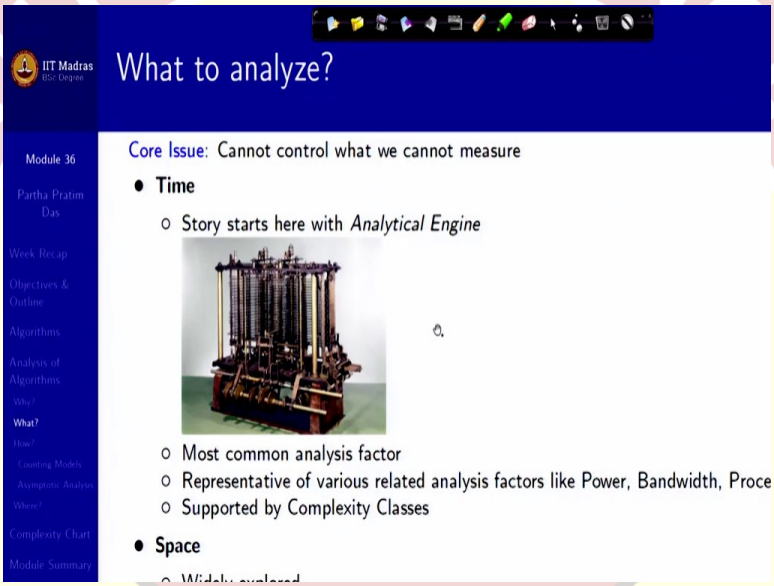


So, why you analyze? Because resources are scarce. I mean, because we always want to do more with less. We greed for, do more with less. We want to avoid performance bugs. So, these are the reason we will try to analyze. So, the core issues is to predict performance, as to how much time does binary search tree take, or binary search take if I have two algorithms for the same task. Like, for sorting an a set of data in an array, you have innumerable algorithm, Quicksort, Mergesort, (Buck) Bucket Sort, Insertion Sort,

Selection Sort, Heap Sort, you, you name it. So, you need to compare the algorithms and decide which one should we use.

Then you want to prove, provide guarantees, that okay, if I do it in this way, if I use a Red-Black tree then I would always be able to insert a key, insert a record in my database in order n log n type, whatever that means. We want to understand the theoretical basis. That is what I was saying, lower bounds and all that. So, there are these are the primary reasons why you analyze.

(Refer Slide Time: 09:02)



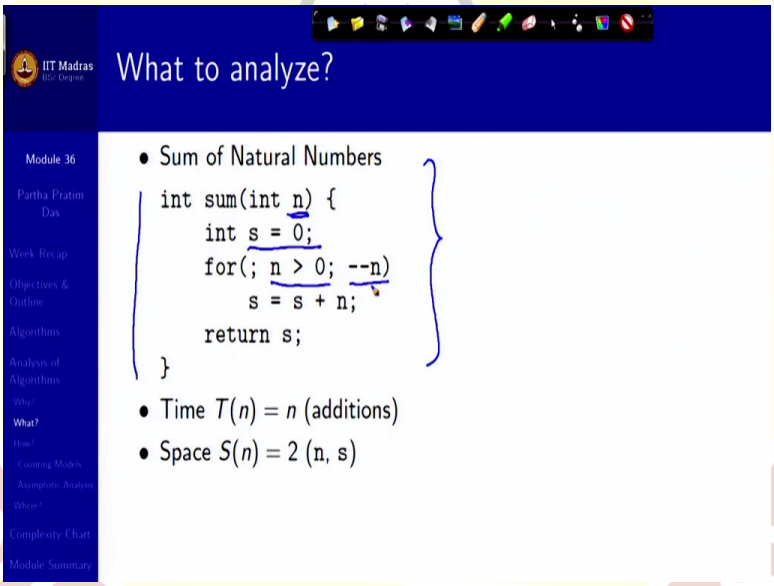
Now, the question is what you analyze? You analyze for resources taken, given an input size. So, your parameter is the input size, which could be the number of elements in the input, which could be the size of the input, which could be the value of the input, which could be multiple inputs, but that is your parameter. And what do you measure? What do you want to measure, is the resource which is important for you.

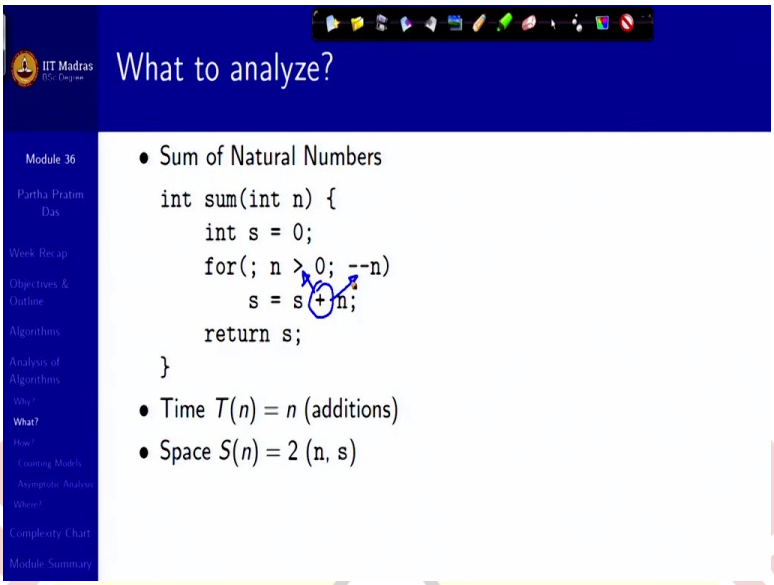
So, the whole history of computing analysis started with analysis of time. It started with the analytical engine of which I have just put the picture here, is fun, so the most common analysis factor is the time, but there could be several others. For example, storage. Space is also very critical in many places, for example, in handheld devices, in databases because databases are very, very large.

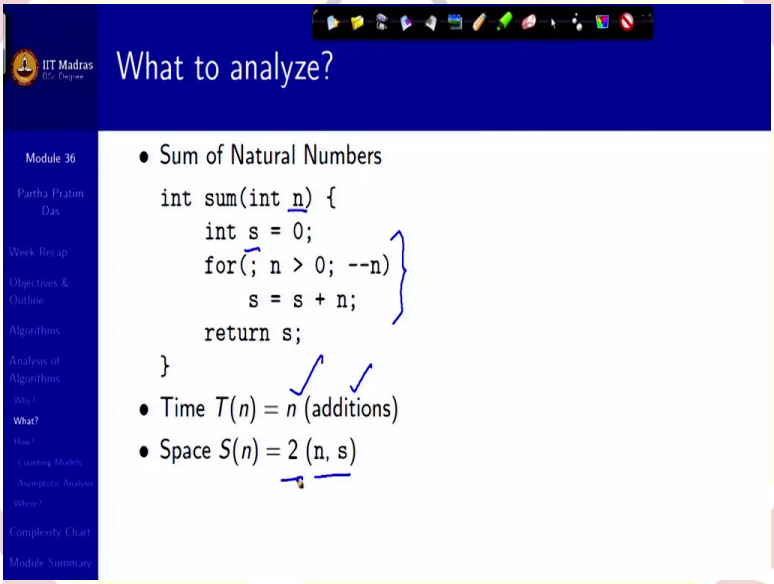
So, if you can save some even 10 percent space, you save a lot of money. But there could be other factors to analyze also. For example, power I think I mentioned it earlier as well, that handheld devices are power, I mean, power constrained, so you need to possibly trade off other resources in favor of power. You may want to optimize for bandwidth because you may not have enough bandwidth, a 4G bandwidth to connect to your server. You may have a 2G bandwidth.

So, you may want to minimize on processor resources in a extremely multi-threaded distributed application and so on. So, there are various different things to analyze. In this context, for the context of database that we are targeting we will only focus on the time and space for now.

(Refer Slide Time: 10:54)







So, what you analyze, this is just taking an example. This is a very standard algorithm for, written in terms of a C program, for making the sum of n numbers. So, this n is your input size. So, that is the parameter. And what do you measure? You want to measure how much time it will take.

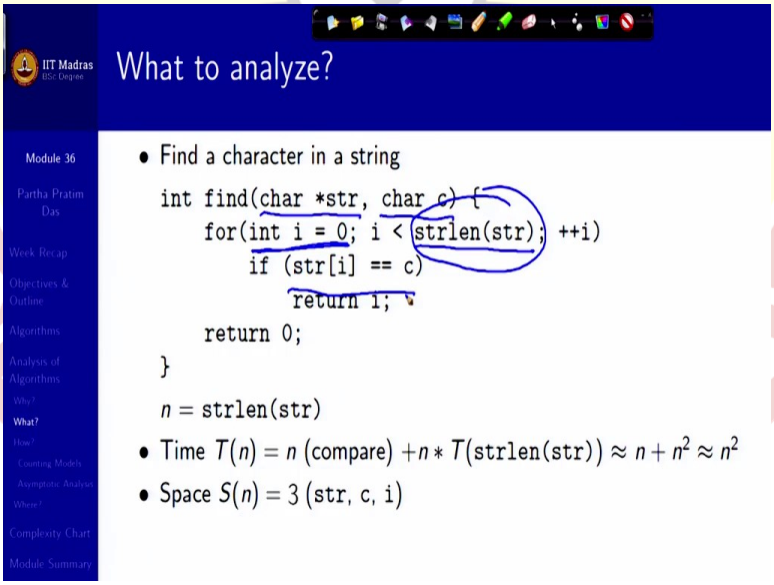
Now, you cannot measure each and everything in an algorithm because there is a time to create the stack for this particular function, there is time for making this assignment, there is time for taking the comparison, doing this decrement and so on. But what you try to identify are the key tasks, key operations that you must do in doing this algorithm.

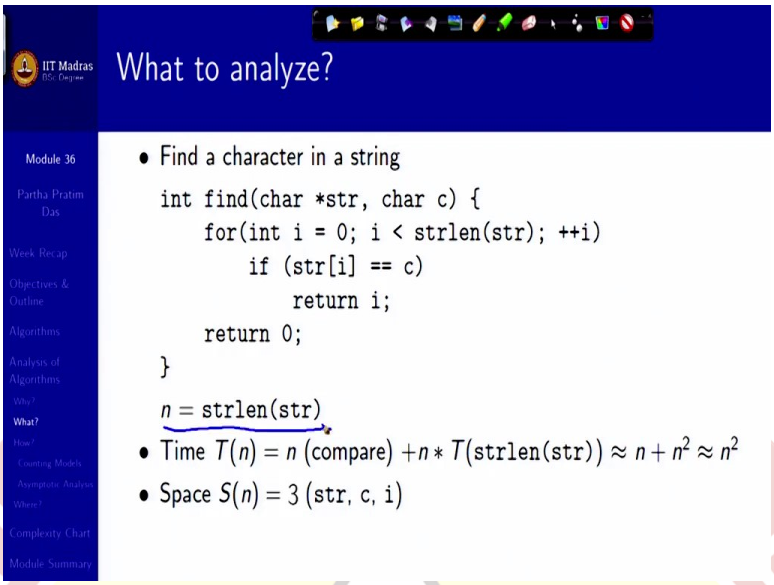
And that key here, since you are trying to find out sum, is the addition operation. And you can also kind of reason that the number of times you need to add, your number of comparisons and number of decrements, here will be related to that because every time you go through the for loop you do a comparison, you do a decrement. And if it succeeds you do an addition.

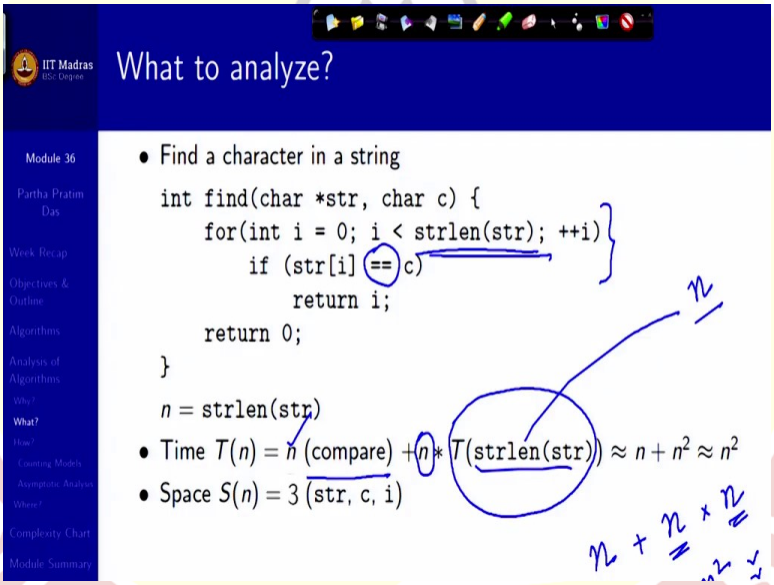
So, these are very closely related. So, approximately, I can say that if I know the number of additions being done, I know how much time will it take the number of additions times the typical time of doing an addition multiplied by a maybe a scale factor. So, if I just count that this loop will go over n times so the time is Tn, which is n additions.

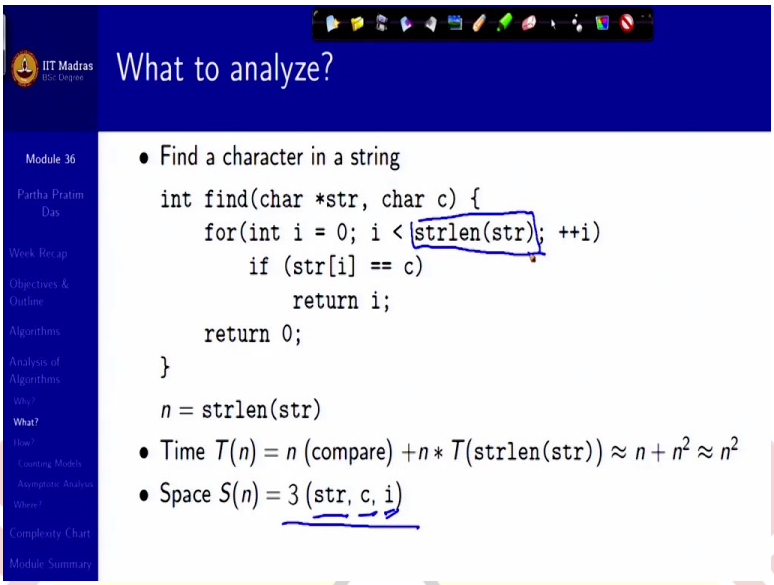
And what is the space? Space is simply the n where you keep the number and the storage s. Of course, there is additional space in terms of temporaries and all that, which we grossly ignore in terms of doing this analysis because it turns out to be comparable to the user variables that you have. So, you have two variables so space is two. So, this is, this is what you analyze.

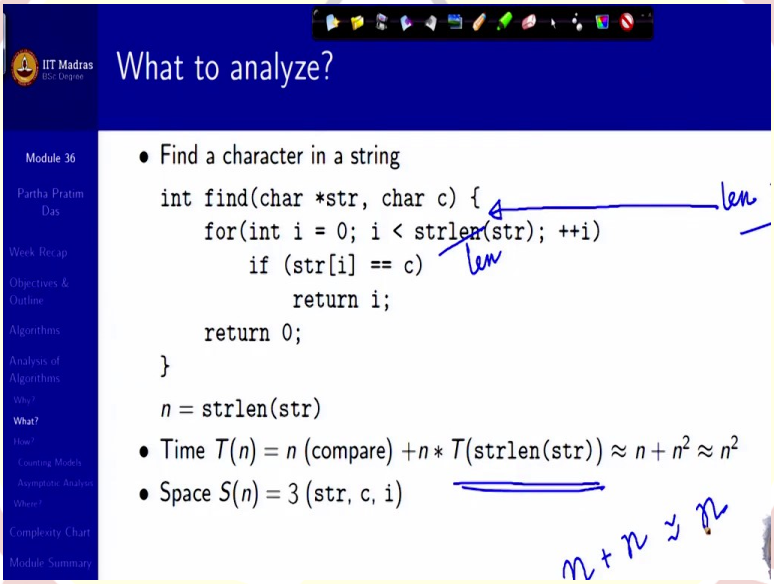
(Refer Slide Time: 13:02)




IIT Madras
BSc Degree
What to analyze?
Module 36
Partha Pratim Das
Week Recap
Objectives & Outline
Algorithms
Analysis of Algorithms
Why?
What?
How?
Counting Models
Asymptotic Analysis
Where?
Complexity Chart
Module Summary
• Find a character in a string
int find(char *str, char c) {
for(int i = 0; i < strlen(str); ++i)
if (str[i] == c)
return i;
return 0;
}
n = strlen(str)
• Time T(n) = n (compare) +n * T(strlen(str)) ≈ n + n² ≈ n²
• Space S(n) = 3 (str, c, i)


IIT Madras
BSc Degree
What to analyze?
Module 36
Partha Pratim Das
Week Recap
Objectives & Outline
Algorithms
Analysis of Algorithms
Why?
What?
How?
Counting Models
Asymptotic Analysis
Where?
Complexity Chart
Module Summary
• Find a character in a string
int find(char *str, char c) {
for(int i = 0; i < strlen(str); ++i)
if (str[i] == c)
return i;
return 0;
}
n = strlen(str)
• Time T(n) = n (compare) +n * T(strlen(str)) ≈ n + n² ≈ n²
• Space S(n) = 3 (str, c, i)





So, if we take another example quickly, this is finding a character in the string. So, what you are doing is, you are writing a, this is, the pointer to the string, this is the character that you want to find, and you go over that string starting with the index 0, so you are trying to check str 0, str1, like this, and how far do you go? You go up to strlen(str), which gives you the length of the string.

So, those many characters are there, those many times you have to do. So, those many times this loop will have to execute, and for each c, if it, if the character is c, equal to c or not. If it is equal, you return, you are done. If it is, not then you return a 0, saying that you did not get it.

Now, given this, so your, what is the input complexity? Input complexity is the size of the string. The longer the string, you need more time. So, your complexity parameter is n. Now, what is that you get to do? Naturally, again the key, you are trying to see for match, so the key operation is compare. So, you have, and this loop can go on variable number of times depending on where you find the character.

So, what you try to say? You try to say, okay, what is the maximum possible, what is the worst possible scenario I can get into, which we will discuss in subsequent slides also, and for that, you know that if the character does not exist at all, then you will have to go through the entire string. So, you said that it could be at the worst, it could be n comparisons, n comparisons.

Now, every time you go over this loop, you do strlen(str). So, it also has the, n times it has to do strlen(str), so whatever time that takes, you will also have to take that. Now, what is strlen(str)? It has to count, going one after the other. There is no other way. You have to count or keep on seeing, keep on incrementing an index till you get a null character which is a c string terminator.

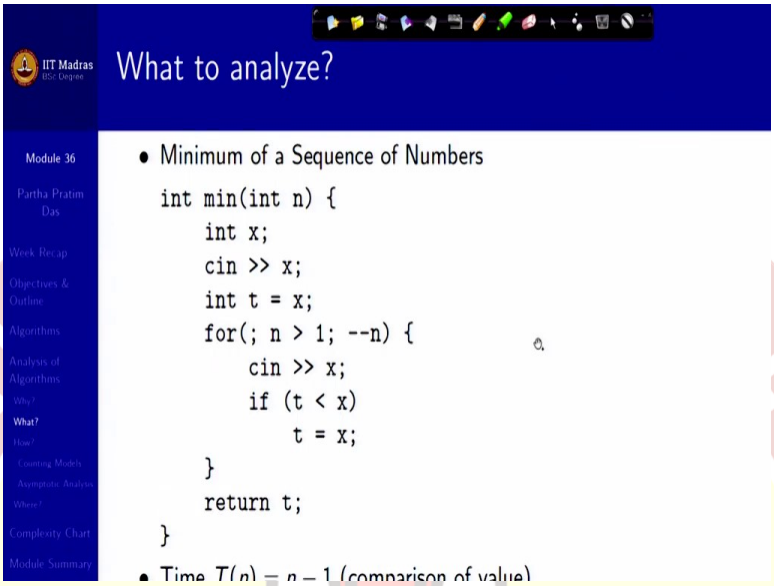
So, every time you do this you have n. So, what you get, n comparisons plus n times n increments because you have to go through that index, and every time you go through the loop you have to do this. So, it turns out to be n plus n square. And if n square starts going large then I can say this is approximately n square. So, we will say this is a quadratic algorithm.

What is your space? Your space requirement is the, input is str and i, c and you have just the i. So, space requirement is still very negligible. Now, that is where you can see the benefit of doing the analysis. For example, you are just checking in the string. So, the string does not change as you go over the loop. So, strlen(str), value that you will get, the first time, the second time, the third time, the nth time, will all be the same.

Then you are basically losing efficiency but, by computing it several times unnecessarily. So, what you could do? You could remove this and before that loop you could say len assigned strlen(str). And make this length. Then what will happen? This will get computed in n, and you will not need this strlen time after that. So, this will turn out to be n+n, which is, which will be n. So, from quadratic it will become linear. A very simple

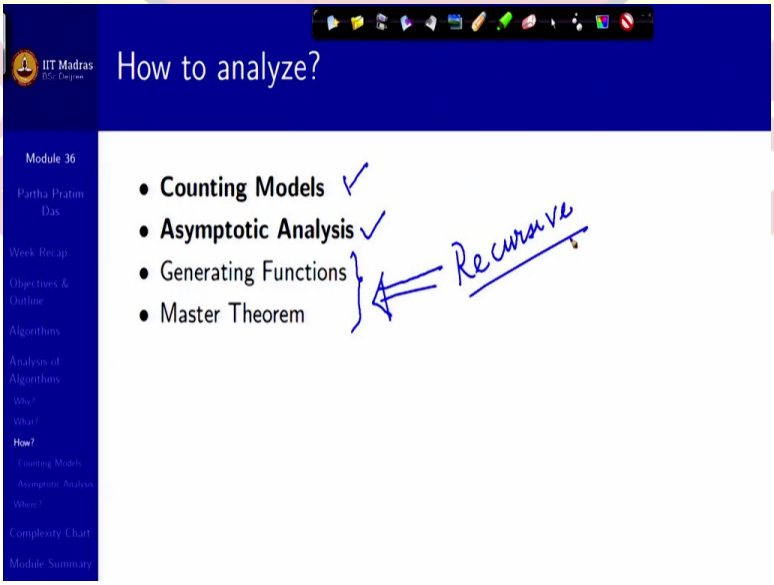
one statement change can drastically improve your algorithm and that is what you analyze for the time and the space.

(Refer Slide Time: 17:23)



So, there are a couple of more examples, that I have provided here, like finding the minimum of a sequence of numbers like in, in different ways so that you can see how you are actually getting the improvement, and you can go through them and practice. So, this is what, this is what you look for.

(Refer Slide Time: 17:37)

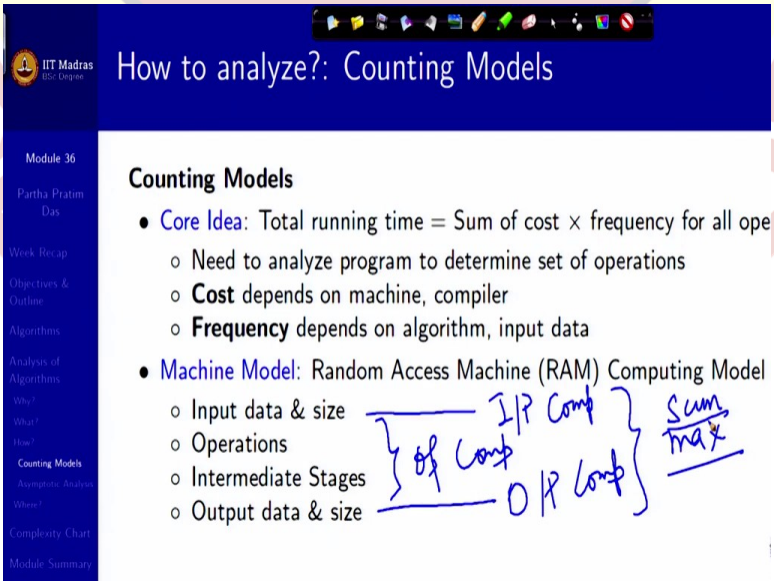


Now, the question, question is how do you do it? How do you given a, given an algorithm or a program source, how would you do the analysis? It is not a trivial question. Now, there are different models. One is counting model. Like, whatever we were doing is a counting model. We are just observing and seeing, okay this is, this is, what likely, likely to be you know more dominant, let us count how many times it can happen. It is not always easy to do that.

Also, given the, given the, all different kinds of instruction you could have, the counting could turn out to be a very complex formula. So, we do something of an approximation which we say is an asymptotic analysis. So, these are some of the basic methods. And then you have, particularly since several algorithms are recursive, it is very difficult to do accounting reasoning on them.

So, what you do, you write corresponding recurrence relations, use generating functions to solve them and get an idea about the complexity number, or there is, for a large class of problems, large class of recursive decomposition, you have a basic theorem known as the masters theorem. So, this is dominantly useful for recursive algorithms and, and since we are not doing an algorithms course I am keeping this aside because we just need the basic results to go forward.

(Refer Slide Time: 19:19)



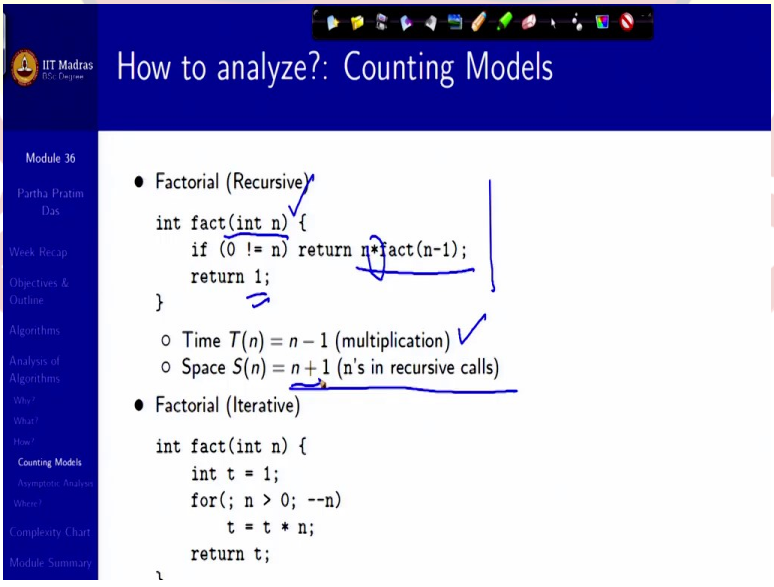
So, we have already seen some part of the counting model where we try to find out the frequency of different operations, get an estimate about the cost and then your total

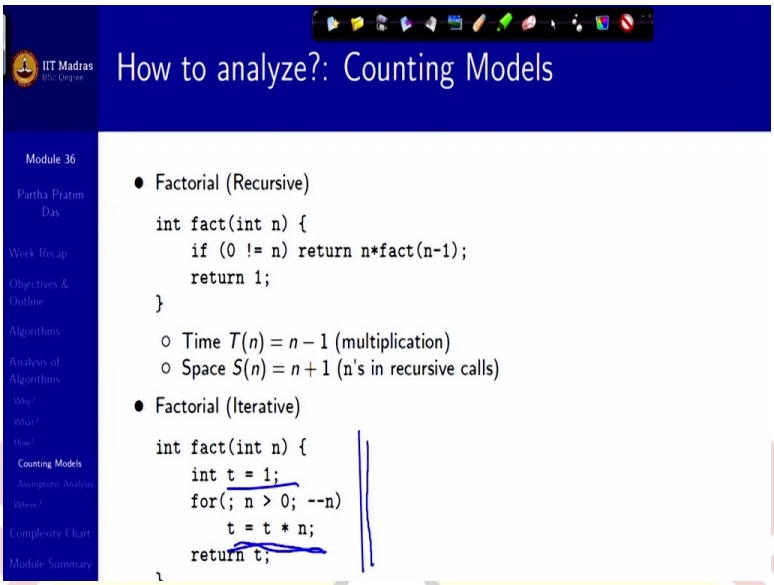
running time, or total space, total running time is a sum of cost times frequency for all operations. And for this, we use a simple random-access machine model where we know the input data and size, we can identify the operations which are known a priori, and we can identify the intermediate stages, and we look at output data and size.

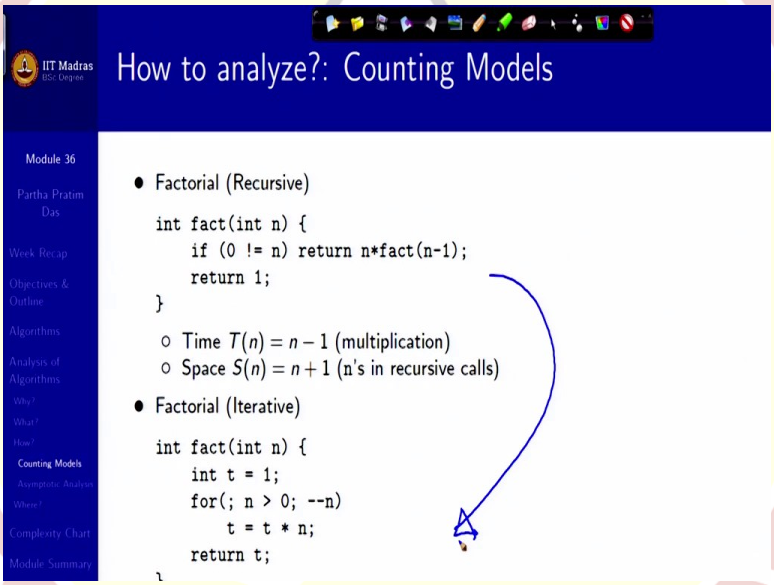
And the maximum of these, that is certainly you will need to read the input. Not certainly, in most cases. There are problems where you do not need the need to read the entire input. And you will need to write the output, and you will need to do the operations. So, the maximum of these three will give you the complexity, and that is the basic approach to analysis because if you have to read the input then you cannot have a complexity less than the input size. If you have to write the output then you cannot have an complexity which is less than the output size.

So, the first one we call input complexity, based on this, I am sorry, this is called input complexity, this is called, because you will have to write the output, in any case, complexity, and whatever operations complexity. So, the max of these, or basically the sum of these, I am saying max because something will dominate anyway, so, is the total complexity. And that is how you, you analyze.

(Refer Slide Time: 21:06)







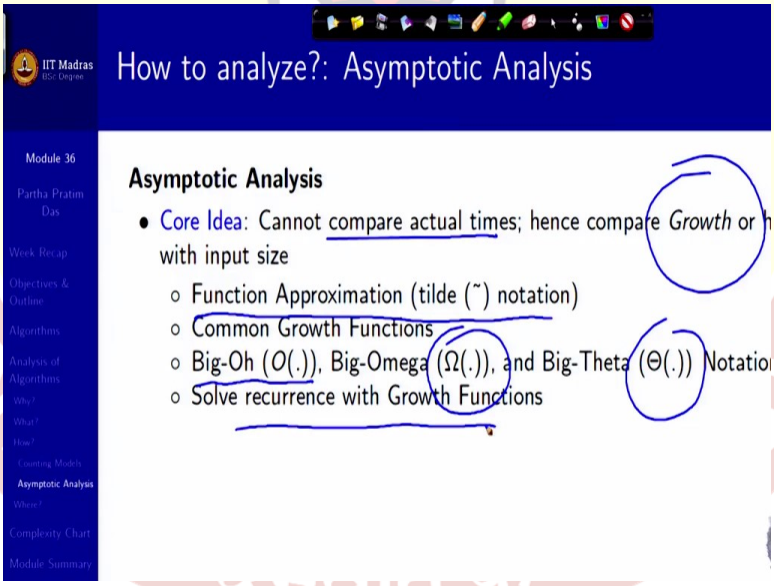
So, this is just a factorial example. This is written recursively. I am not using generating function, but it is very easy to see that you have to find factorial n, it is n times factorial n minus 1 so this will go down to factorial 0 or factorial 1, factorial 0, which I know explicitly as 1.

So, n times this call will be made. It is very clear. So, n times a call will be made means this multiplication will be done n-1 times because the first call with n does not have a multiplication. So, it is n-1 multiplication but it needs a lot of space because as you keep on calling none of these functions are returning.

So, the call stacks, it is called I mean, I am sorry, activation records or stack frames keep on staying on the stack so there are, n+1 stacks the, the, the caller, the n invocations of the factorial will be there. So, there will be, the space will be of the order of n+1 times a constant because every stack frame will take a size. n-1 is the number of multiplications, times a constant is your time.

But if you write this simply in an alternate iterative form where you start not from n down to 1 but you just keep on multiplying from 1 up to n, if you just do that, as in here, then obviously again you will have n multiplications but you will just need one function call, in this function itself. So, all that you need is n and t. So, you can see that just moving from this to this you make a great advantage by reducing your space significantly. So, this is, earlier we showed an example where you were able to reduce your time, here is an example where you can show that you can reduce your space. So, this is how you analyze typically in terms of a counting model.

(Refer Slide Time: 23:13)



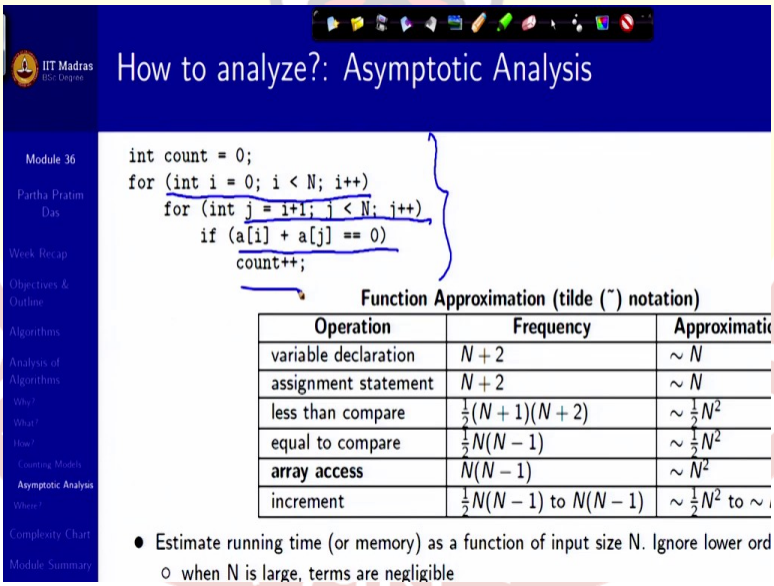
Now, the core idea is that we cannot often just do comparison in terms of whether it is n or n+5 and things like that. The comparing actual times is not a good idea, it is not possible. The actual time will depend on the processor resource, I mean, processor clock speed, it will depend on the particular processor architecture, it will depend, depend on the bus speed, and so on so forth.

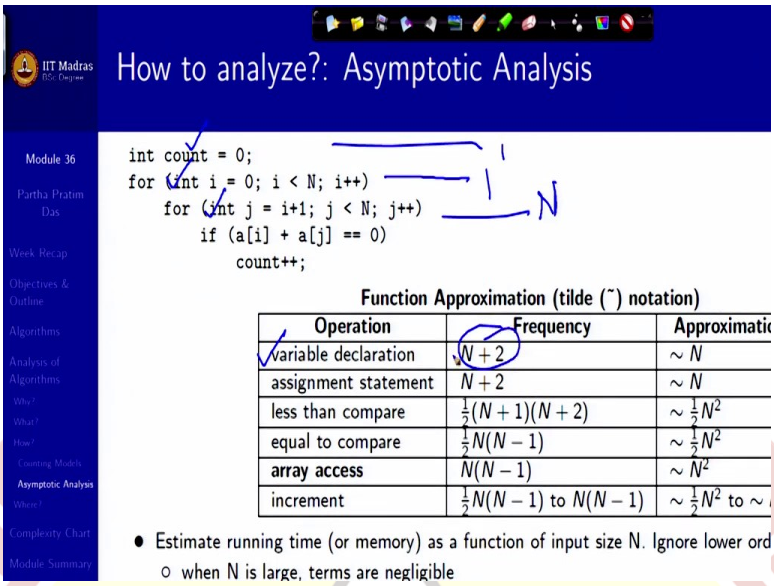
So, comparing actual, I am sorry, comparing actual time is not an option. So, what you compare is growth. That is what hurts you, because you do not, if n is small, if your input size is small you really do not care how much time a database table will take to look it up because it will be less anyway, whatever way you do it.

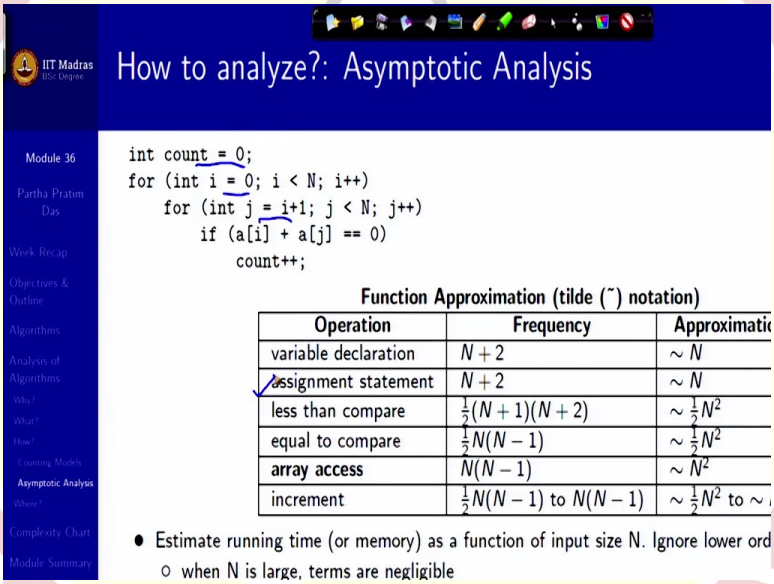
If n is large, you really care. So, you want to know that if you keep on increasing n from a small value to a large value, how is your time and space going to grow. It is the growth, that is what you are interested in. And you are not interested in a specific number or a specific function because you cannot compare them.

So, you want a approximation of that, and that is what is known as the function approximation, which we call, which we denote by a Big-Oh notation. There are other complexity notation, like Big-Omega, Big-Theta, which relates to the theory of lower bounds, so we are not getting into those. Often you solve a recurrence in the growth function to, to get the growth function. But we will just show you simple examples.

(Refer Slide Time: 25:02)




IIT Madras
BSc Degree
How to analyze?: Asymptotic Analysis
Module 36
Partha Pratim Das
Week Recap
Objectives & Outline
Algorithms
Analysis of Algorithms
Why?
What?
How?
Counting Models
Asymptotic Analysis
Where?
Complexity Chart
Module Summary
int count = 0;
for (int i = 0; i < N; i++)
for (int j = i+1; j < N; j++)
if (a[i] + a[j] == 0)
count++;
Function Approximation (tilde (~) notation)
Operation
Frequency
Approximati
variable declaration
N + 2
~ N
assignment statement
N + 2
~ N
less than compare
1/2(N + 1)(N + 2)
~ 1/2 N^2
equal to compare
1/2 N(N − 1)
~ 1/2 N^2
array access
N(N − 1)
~ N^2
increment
1/2 N(N − 1) to N(N − 1)
~ 1/2 N^2 to ~
Estimate running time (or memory) as a function of input size N. Ignore lower ord
when N is large, terms are negligible


IIT Madras
BSc Degree
How to analyze?: Asymptotic Analysis
Module 36
Partha Pratim Das
Week Recap
Objectives & Outline
Algorithms
Analysis of Algorithms
Why?
What?
How?
Counting Models
Asymptotic Analysis
Where?
Complexity Chart
Module Summary
int count = 0;
for (int i = 0; i < N; i++)
for (int j = i+1; j < N; j++)
if (a[i] + a[j] == 0)
count++;
Function Approximation (tilde (~) notation)
Operation
Frequency
Approximati
variable declaration
N + 2
~ N
assignment statement
N + 2
~ N
less than compare
1/2(N + 1)(N + 2)
~ 1/2 N^2
equal to compare
1/2 N(N − 1)
~ 1/2 N^2
array access
N(N − 1)
~ N^2
increment
1/2 N(N − 1) to N(N − 1)
~ 1/2 N^2 to ~
Estimate running time (or memory) as a function of input size N. Ignore lower ord
when N is large, terms are negligible

How to analyze?: Asymptotic Analysis


```
int count = 0;
for (int i = 0; i < N; i++)
    for (int j = i+1; j < N; j++)
        if (a[i] + a[j] == 0)
            count++;
```

Function Approximation (tilde (~) notation)

Operation	Frequency	Approximati
variable declaration	$N+2$	$\sim N$
assignment statement	$N+2$	$\sim N$
less than compare	$\frac{1}{2}(N+1)(N+2)$	$\sim \frac{1}{2}N^2$
equal to compare	$\frac{1}{2}N(N-1)$	$\sim \frac{1}{2}N^2$
array access	$N(N-1)$	$\sim N^2$
increment	$\frac{1}{2}N(N-1)$ to $N(N-1)$	$\sim \frac{1}{2}N^2$ to $\sim$

- Estimate running time (or memory) as a function of input size N. Ignore lower ord
  - when N is large, terms are negligible

IIT Madras BSc Degree

How to analyze?: Asymptotic Analysis

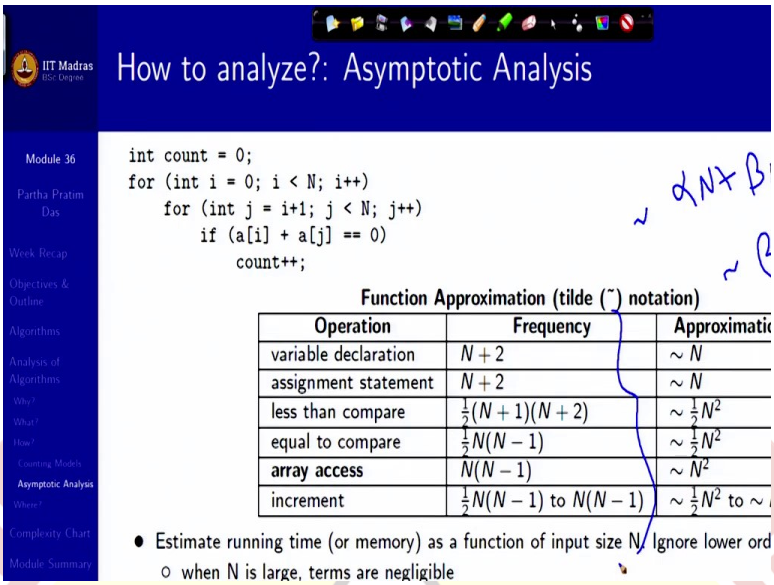
Module 36
Partha Pratim Das
Week Recap
Objectives & Outline
Algorithms
Analysis of Algorithms
Why?
What?
How?
Counting Models
Asymptotic Analysis
Where?
Complexity Chart
Module Summary

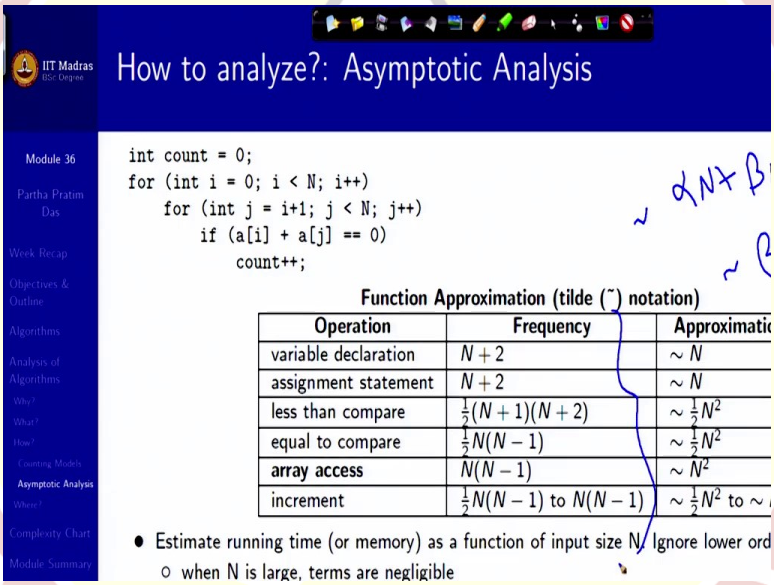
```
int count = 0;
for (int i = 0; i < N; i++)
    for (int j = i+1; j < N; j++)
        if (a[i] + a[j] == 0)
            count++;
```

Function Approximation (tilde (~) notation)

Operation	Frequency	Approximati
variable declaration	$N+2$	$\sim N$
assignment statement	$N+2$	$\sim N$
less than compare	$\frac{1}{2}(N+1)(N+2)$	$\sim \frac{1}{2}N^2$
equal to compare	$\frac{1}{2}N(N-1)$	$\sim \frac{1}{2}N^2$
array access	$N(N-1)$	$\sim N^2$
increment	$\frac{1}{2}N(N-1)$ to $N(N-1)$	$\sim \frac{1}{2}N^2$ to $\sim$

- Estimate running time (or memory) as a function of input size N. Ignore lower ord
  - when N is large, terms are negligible





Let us say, this is an example of, there is, there is, what is this trying to do is, is going from 1 to n, you have an array a, and then for every i it is going from i+1 to n and it is checking. So, it is basically checking on combinations to check if the sum of any two elements is 0 or not, or how many times it is it is 0, how many such sums are 0. So, that functionality you do not really bother about.

Now, the question is, if we do a counting model on this, then you will see that, how many, what is the frequency of that? You have it one, so I am just trying to look at the variable declarations, the number of times you will have to hit at the declaration. One is

here, one is here, which will happen once, and this will happen multiple times, every time you go, do the outer for loop you will start the inner for loop.

So, you have a declaration to process. So, the outer for loop will happen n times, so the inner for loops int j declaration has to be processed n times. So, this is 1, this is 1, and this is n, which makes the total as n+2. The same thing can be said about the assignments.

So, these are the assignments. Actually, we call it initialization, but in here, it will act as an assignment, which is n+2. How many times you do less than comparison? This is this because you do it, for every time you do it for the outer, you do for the i times for the inner. So, it is, it is 1+2 plus like, 3, sum of natural numbers. So, it is what it is.

Equal to comparison will happen as many times the inner loop can happen, inner loop iteration can happen. This is, this will be the array axis, double of that, two times of that because you have two axes here, and increment will depend on what are your data. It will not happen always. These increments are fixed but the increment here would vary depending on the data. So, this is, this is what you get. So, this is your total set of analysis. So, if you put, sum all of them together you will get a complex quadratic, you can see that n square is a maximum, so you will get a quadratic expression. That is fine.

But what we do? We do we do an approximation. We say that the growth of n+2 and the growth of n are similar. They do not significantly differ in the growth. They will grow in a similar way. Similarly, the growth of this expression and the growth of half n square, or for that matter n square is similar. They will just differ by a small factor but the growth will be the similar.

So, in this way, we, what we do is in simple terms, if it is a, these, these all happen to be polynomials, so what we are doing is in simple terms, we just take the dominant term, the highest degree term and say that is my tilde(~) approximation, or asymptotic approximation. So, all that you get, all are these approximations. So, what I do, I add up all of them. So, I end up having alpha n times beta n square, where alpha beta will get values. So, this is the approximation.

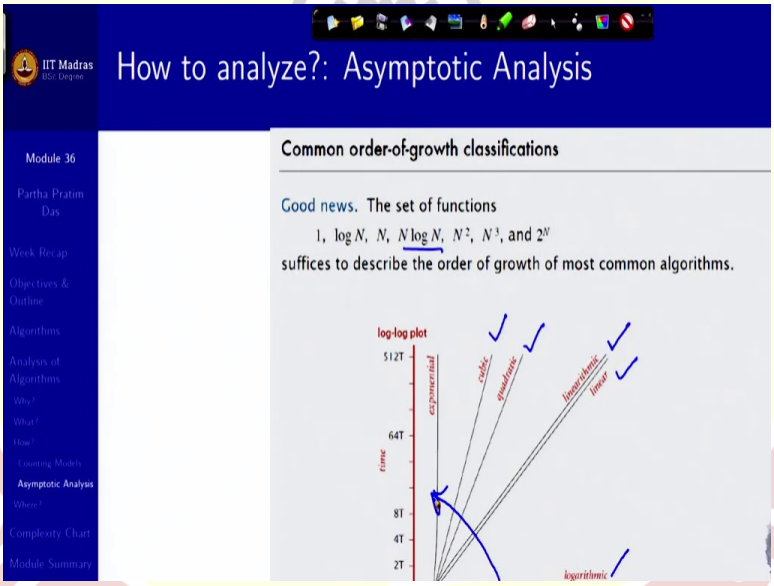
And then we say that well n square will grow much faster than n. n is linear, n square goes like this. So, I can ignore, for large n, I can ignore alpha n because if n is small they

are comparable, and I do not care what happens, with the small input size I do not care what is the complexity.

If it grows large, then this, this gap keeps on increasing. This gap keeps on increasing. So, what matters is the quadratic term, beta n square. So, I can say that alpha n plus beta n square, what is the tilde(~) here is actually beta n square, ignoring the linear term. If I say that, then I can say that, that is actually n square because beta is just a constant factor.

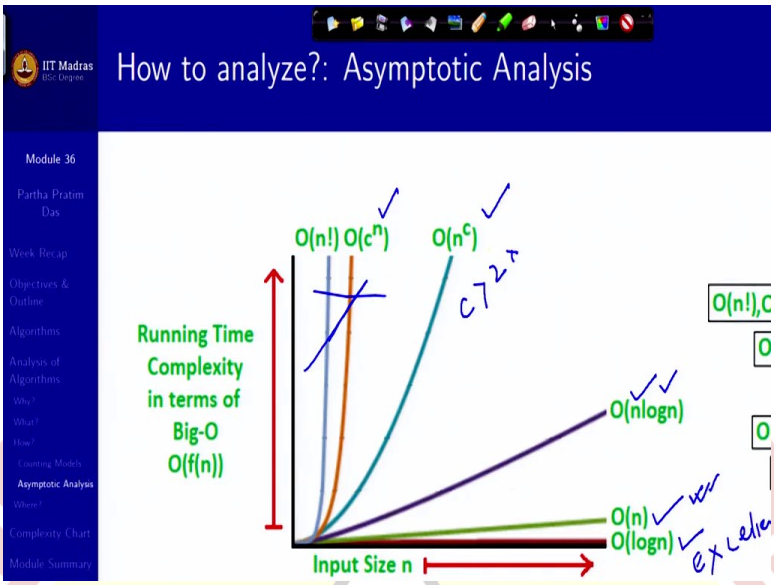
So, that is the whole idea of saying that it is asymptotically n square. We will write this as order n square. What this tilde function really means is f is approximation to, g is an approximation to n if the limit of f by n, n tending to infinity, is 1. And you can check in every case here that that limit actually is what holds.

(Refer Slide Time: 30:37)



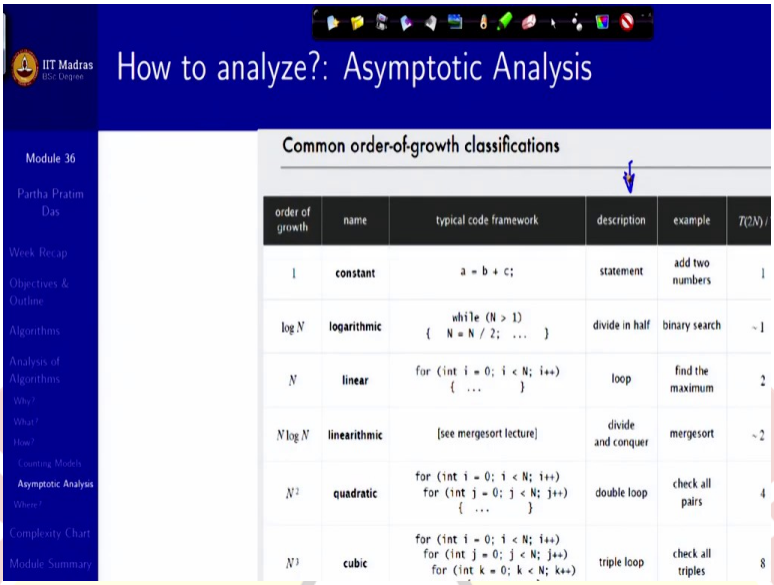
So, this is just a chart to show you how does the typical known complexities of 1, which is constant, log n, which is here, logarithmic linear, linear logarithmic, that is, n log n, quadratic, cubic, you may not be familiar to see these curves in this form because it has been plotted in a log log curve. Your x axis is log, as well as right axis is log. So, that as you grow in the polynomial basically your line always remains same, you are just changing the gradient. It is just rotating like this and that tells you how the cost is increasing.

(Refer Slide Time: 31:24)



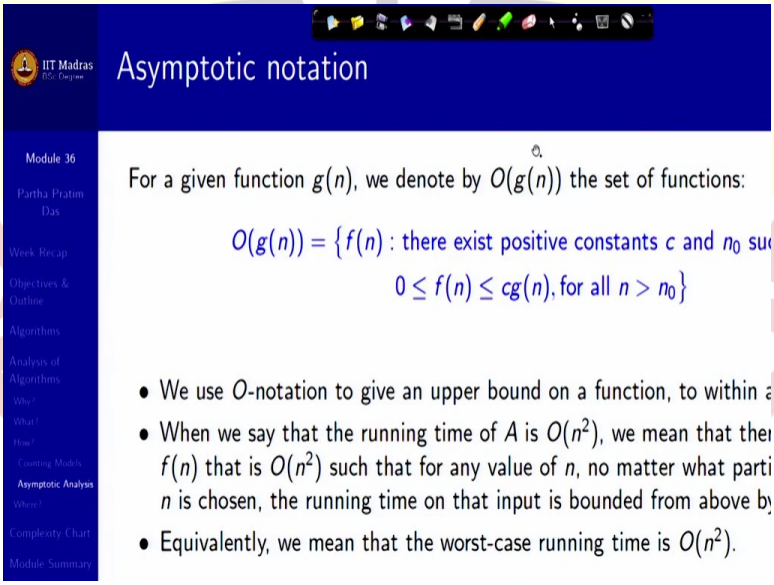
If you would like to look at it in your familiar graph plotting, then this is how it will look like. So, you can see that if something is order n it goes like a straight line, if something is log n it is much less, almost like constant. n log n goes like this, but powers of n go very far, fast. Exponential of n are almost vertical, that with a very small n, it explodes. So, you cannot, these are what you cannot afford. This is, maybe c>2 should be avoidable. This is preferred, this will be good, and this is excellent. So, that is what, along this axis, that is what we will thrive for. And that is the basic take back from this.

(Refer Slide Time: 32:13)



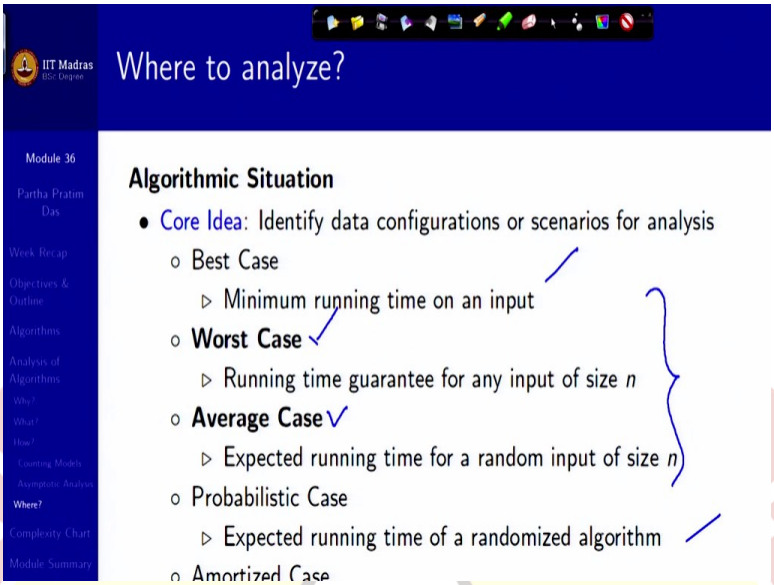
Here, I have given a list of these complexities and typically a description. Just focus on this description when you study to know as to where you can typically find this complexity.

(Refer Slide Time: 32:30)



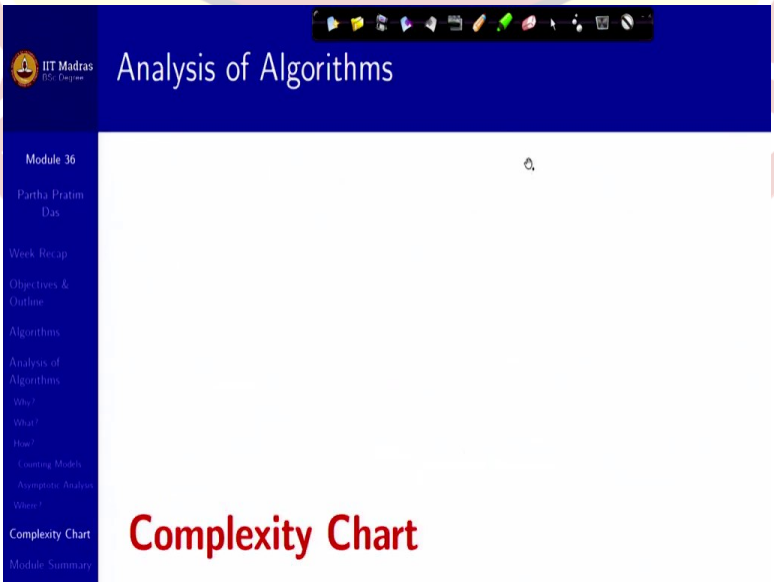
So, this is the formal definition. I am not very concerned about the formal definition. You have done it in the algorithms course. This is just for your reference.

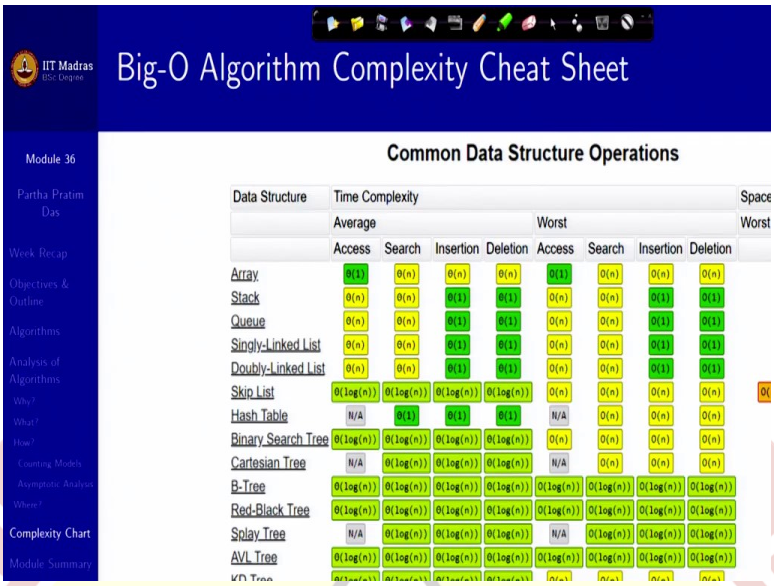
(Refer Slide Time: 32:38)



And finally, what do we, where do we do the analysis? There could be different situations, but we will focus primarily on these two. The worst case, that is, if I take all possible cases what is the input? That gives me the worst complexity. And also, if I take a any random input, assuming inputs are distributed in a certain distribution, then what is the expected running time we will get. We will typically talk of these two. These are other forms of complexity analysis that exist but primarily databases or things used in databases, we will primarily focus in the worst case and the average case.

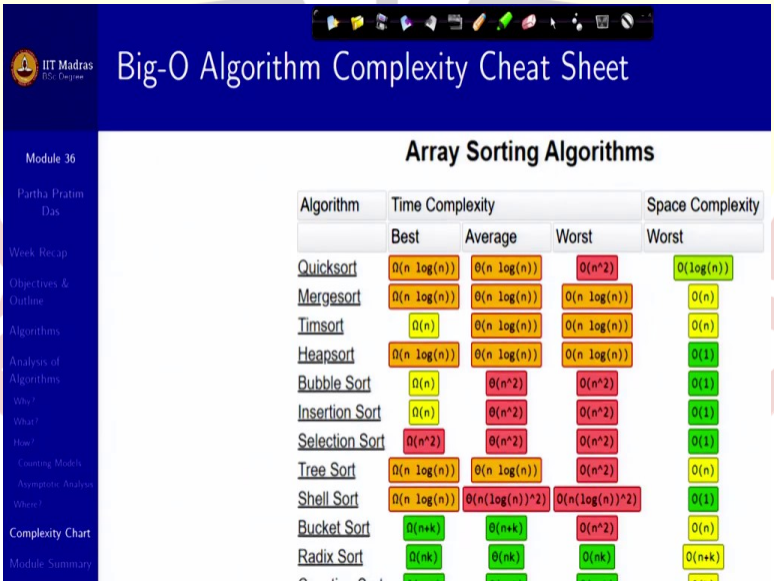
(Refer Slide Time: 33:24)





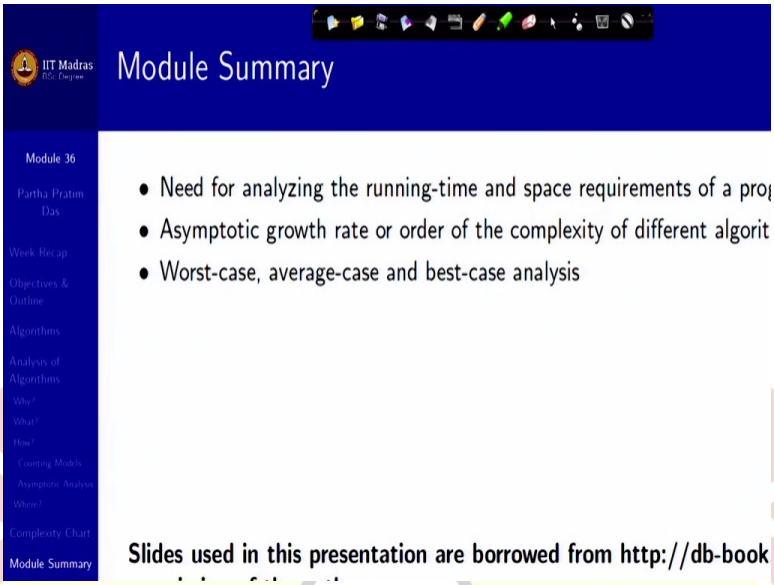
So, I will end with a complexity chart, this is not for you to remember but to refer, where for the common data structure, some of these I am going to discuss in the next two modules. What are their different access times? So, if you, this is just a quick reference that you can make.

(Refer Slide Time: 33:41)



And another is for the sorting algorithms. What are the different complexities you get in the different scenarios, both in terms of time as well as in terms of space?

(Refer Slide Time: 33:52)



So, with that quick introduction, or the, I mean, I should say the revision of algorithms and complexity, we conclude this module. Thank you very much for your attention and we will meet in the next module and talk about some of the data structures.